

FRISBEE

Source Code Design Notes

Personal Behavioral Alignment Modeling via Logistic Regression on Free-Form Text

Internal reference — not a repeat of the README or slides

Why the code is structured the way it is

FRISBEE has one unusual constraint: every piece of ML and storage must run entirely on the user's device, with no server, no cloud model call, and no data leaving the browser. That constraint drives every architectural decision below.

The codebase splits into two concerns that must never couple directly: the **ML pipeline** (`src/lib/yours/`) and the **UX reaction layer** (mascot, sounds, haptics). They communicate only through the event bus. This is why you cannot find a direct import from `buddy.ts` into `engine.ts` or vice versa.

The ML pipeline in reading order

tokenize.ts — why apostrophes matter

A naïve tokenizer splits on whitespace and punctuation, turning *didn't* into *didn* and *t*. Both fragments miss the cluster entirely. The apostrophe-preserving tokenizer keeps contractions intact because contractions are disproportionately common in the *excuse* and *drift* clusters: *couldn't*, *didn't*, *wasn't*. Losing them degrades the most diagnostic cluster.

features.ts — the blended IDF decision

The feature vector $\mathbf{x} \in \mathbb{R}^4$ is built by scoring each of the four semantic clusters via TF-IDF. The IDF component uses a **blended formula**:

$$\text{IDF}_{\text{blend}} = \alpha \cdot \text{IDF}_{\text{personal}} + (1 - \alpha) \cdot \text{IDF}_{\text{background}}$$

The **reason for blending** is the cold-start problem. In the first week, the user has written one message. Every word in that message has personal document frequency = $1/1 = 1$, so $\text{IDF}_{\text{personal}} = \log(1/1) = 0$ for every word. The feature vector collapses to all zeros and the model cannot update. The background IDF from `vocab.json` seeds the IDF with plausible weights from day one. As weeks accumulate, α (personal weight) can grow so the model becomes progressively more yours and less generic.

Without the blend, FRISBEE would be useless for the first 3–4 weeks, which is exactly when a new user is deciding whether to keep using it.

model.ts — the regularization asymmetry

The update rule is:

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} - \eta [(\hat{p} - y) \mathbf{x} + \lambda \mathbf{w}] \\ b &\leftarrow b - \eta (\hat{p} - y)\end{aligned}$$

The L2 penalty ($\lambda = 0.01$) is applied to $\mathbf{w}$ only, not to b . This is intentional. Regularizing the bias would systematically pull predictions toward $p = 0.5$ regardless of evidence. The bias captures the user's base rate of yes-weeks (e.g., if someone labels 70% of weeks as yes-weeks, the bias should reflect that). Penalising it would destroy that signal. Regularizing $\mathbf{w}$ prevents any single cluster (e.g., *energy*) from dominating when data is sparse.

scientist.ts — the interpretability layer

This module is architecturally a side-branch, not part of the prediction hot path. It does not affect $\mathbf{w}$, b , or $\hat{p}$. Its job is **word-delta hypothesis surfacing**: it looks for words that appear frequently

on good weeks and rarely on bad weeks (or vice versa) and surfaces them as hypotheses via the event bus.

The reason it is decoupled: hypothesis surfacing is computationally heavier and runs at a different cadence from prediction. It fires an event (`hypothesis:confident`) only when it has enough evidence to be non-trivial. The mascot reacts to that event with a smug line. Neither the ML engine nor the mascot brain knows anything about the other.

The event bus

Why pub/sub instead of prop-drilling or a global store

The app has four independent reaction channels: the mascot (visual mood + voice), haptics, sounds, and score tracking. Each needs to react to the same events (message dropped, duel revealed, week committed). If `engine.ts` called into each of these directly it would accumulate four direct imports and create tight coupling that makes any one channel impossible to disable without touching engine code.

The bus (`bus.ts`) has zero dependencies and the full event union is typed as a TypeScript discriminated union:

```
type BuddyEvent =
| { type: "duel:reveal"; truth: 0|1; userCorrect: boolean }
| { type: "hypothesis:confident"; word: string }
| { type: "boot" } | ...
```

Every subscriber gets full type narrowing by checking `e.type` before accessing payload fields. This is why no subscriber needs a runtime cast.

The one real bug in `bus.ts`

The disposer returned by `on()` closes over the local `set` variable:

```
return () => set!.delete(fn);
```

This works because the Set is never replaced — it is created once per event type and reused. If a future refactor replaced the Set (e.g., to support priority ordering), the disposer would silently become a no-op, leaking subscriptions. The safer form is:

```
return () => subs.get(type)?.delete(fn);
```

Not a live bug, but worth fixing before the codebase grows.

The mascot brain

`dominantCluster` — bridging ML to UX

The mascot does not read model weights or probabilities. It reads the feature vector `x` and picks the cluster with the highest score:

```
function dominantCluster(x: [n,n,n,n]): Cluster | "neutral"
```

This is the only point where the ML output becomes a UX signal. A high drift score → the mascot goes *sheepish*. A high energy score → *kachow*. The mapping is intentional: the mascot reacts to your language pattern for the day, not to the week prediction. Prediction is shown separately on `/week`.

The wired flag and HMR risk

`wireBuddy()` uses a module-level `wired` flag to prevent double-subscription. In production this is fine. In development with Vite HMR, the module can hot-reload and reset `wired = false` while the old subscriptions are still alive in the bus's `subs` Map, resulting in duplicate handlers firing for every event. The disposer pattern (the return value of `wireBuddy()`) is the correct fix — it must be called in a `useEffect` cleanup, not ignored.

The voice lexicon

`voice.ts` is compiled with `as const`, which narrows every string array to its exact literal types. This is load-bearing: without it, `pick()` returns `string` and the TypeScript compiler cannot catch missing or misspelled keys. The `pick()` function accepts an optional `seed` parameter for deterministic selection (useful for preventing flicker on re-render), but no call site currently passes one — a latent feature that could be activated for the hypothesis line, which currently picks a random variant each time it fires.

Known limitations worth stating explicitly

Limitation	Note
4-week test period	Four data points cannot support a statistically meaningful MAE comparison. Results are proof-of-concept.
Fixed vocabulary	<code>vocab.json</code> cluster words are hand-curated. A user whose drift language falls outside these clusters will produce weak feature vectors regardless of model quality.
Self-report bias	Ground truth y is user-labeled. Mood at labeling time contaminates the signal — a bad day on Sunday may retroactively make a neutral week a “no-week.”
No confidence display	The model produces $\hat{p} \in (0,1)$ but the UI shows only a binary prediction. Per Hengstler 2020, showing calibrated confidence is necessary for adaptive trust. Currently absent.
HMR double-subscribe	Documented above. Not a production bug but a development irritant if <code>wireBuddy()</code> is not cleaned up properly.

This document covers design rationale not visible from reading the source alone. It is not a substitute for the code, the README, or the slides — it is the layer between them.